

Hands-On Lab: 5G Architecture Modeling with Python

In this lab, we will:

1. **Model synthetic network slices** with different QoS needs.
2. **Simulate resource allocation** using a simple AI-inspired algorithm.
3. **Visualize results.**

1 Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

2 Generate Synthetic Data

We simulate 100 users across **three slices** (URLLC, eMBB, mMTC).

```
np.random.seed(42)

# Slice requirements (latency, bandwidth, connections)
slices = {
    "URLLC": {"latency": 1, "bandwidth": 20, "connections": 30},
    "eMBB": {"latency": 10, "bandwidth": 50, "connections": 40},
    "mMTC": {"latency": 100, "bandwidth": 5, "connections": 200}
}

# Generate synthetic demand
data = []
for s, req in slices.items():
    for i in range(req["connections"]):
```

```
demand = np.random.randint(1, req["bandwidth"]+1)
data.append([s, demand])
```

```
df = pd.DataFrame(data, columns=["Slice", "Demand"])
df.head()
```

3 Resource Allocation Algorithm

A simple proportional allocation algorithm:

- Assign resources proportional to demand.
- Ensure latency-sensitive slices get priority.

```
total_resources = 500 # total available units
```

```
allocation = {}
```

```
# Priority weights: URLLC > eMBB > mMTC
```

```
weights = {"URLLC": 3, "eMBB": 2, "mMTC": 1}
```

```
for s in slices:
```

```
    slice_demand = df[df["Slice"] == s]["Demand"].sum()
```

```
    allocation[s] = (slice_demand * weights[s])
```

```
# Normalize to total resources
```

```
total_weighted = sum(allocation.values())
```

```
for s in allocation:
```

```
    allocation[s] = round((allocation[s] / total_weighted) * total_resources)
```

```
allocation
```

4 Visualization

```
plt.bar(allocation.keys(), allocation.values(), color=['red','blue','green'])
```

```
plt.title("Dynamic Resource Allocation in 5G Slices")
```

```
plt.xlabel("Slice")
```

```
plt.ylabel("Allocated Resources")
```

```
plt.show()
```

This visualization shows how resources are distributed among URLLC, eMBB, and mMTC slices.

Hands-On Lab: 5G Architecture Modeling with Python

In this lab, we will:

1. **Model synthetic network slices** with different QoS needs.
2. **Simulate resource allocation** using a simple AI-inspired algorithm.
3. **Visualize results.**

1 Setup

```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt
```

2 Generate Synthetic Data

We simulate 100 users across **three slices** (URLLC, eMBB, mMTC).

```
np.random.seed(42)  
  
# Slice requirements (latency, bandwidth, connections)  
slices = {  
    "URLLC": {"latency": 1, "bandwidth": 20, "connections": 30},  
    "eMBB": {"latency": 10, "bandwidth": 50, "connections": 40},  
    "mMTC": {"latency": 100, "bandwidth": 5, "connections": 200}  
}  
  
# Generate synthetic demand  
data = []  
for s, req in slices.items():  
    for i in range(req["connections"]):
```

```
demand = np.random.randint(1, req["bandwidth"]+1)
data.append([s, demand])
```

```
df = pd.DataFrame(data, columns=["Slice", "Demand"])
df.head()
```

3 Resource Allocation Algorithm

A simple proportional allocation algorithm:

- Assign resources proportional to demand.
- Ensure latency-sensitive slices get priority.

```
total_resources = 500 # total available units
```

```
allocation = {}
```

```
# Priority weights: URLLC > eMBB > mMTC
```

```
weights = {"URLLC": 3, "eMBB": 2, "mMTC": 1}
```

```
for s in slices:
```

```
    slice_demand = df[df["Slice"] == s]["Demand"].sum()
```

```
    allocation[s] = (slice_demand * weights[s])
```

```
# Normalize to total resources
```

```
total_weighted = sum(allocation.values())
```

```
for s in allocation:
```

```
    allocation[s] = round((allocation[s] / total_weighted) * total_resources)
```

```
allocation
```

4 Visualization

```
plt.bar(allocation.keys(), allocation.values(), color=['red','blue','green'])
```

```
plt.title("Dynamic Resource Allocation in 5G Slices")
```

```
plt.xlabel("Slice")
```

```
plt.ylabel("Allocated Resources")
```

```
plt.show()
```

This visualization shows how resources are distributed among URLLC, eMBB, and mMTC slices.